

```
from google.colab import files
uploaded = files.upload()
```

ReadMe.csv

ReadMe.csv(text/csv) - 125038 bytes, last modified: 6/22/2026 - 100% done
Saving ReadMe.csv to ReadMe.csv

```
# =====
# FULLY EXECUTED NBA FEATURE ENGINEERING PIPELINE (SINGLE CELL)
# =====

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# --- 🛠️ 1. DATA DISCOVERY & LOADING ---
print("🔍 Searching for dataset file...")
csv_file = "ReadMe.csv"

if not os.path.exists(csv_file):
    raise FileNotFoundError(f"❌ Target file '{csv_file}' not found in current")

print(f"📖 Loading data from '{csv_file}'...")
df = pd.read_csv(csv_file)

print(f"✅ Source Data Loaded: {df.shape[0]} players, {df.shape[1]} raw attributes")

# --- 🎯 2. TARGET ISOLATION & NOISE REDUCTION ---
print("🧹 Stripping out non-predictive features and isolating target...")

# Explicit target definition
target_col = 'target_5yrs'
if target_col not in df.columns:
    raise KeyError(f"❌ Target column '{target_col}' missing from dataset.")

# Identify and drop text identifiers / noise indexes to prevent data leak or model overfitting
noise_cols = ['name', 'Unnamed: 0']
existing_noise = [col for col in noise_cols if col in df.columns]

df_cleaned = df.drop(columns=existing_noise)
print(f"    - Removed columns: {existing_noise}")
print(f"    - Isolate target variable: '{target_col}'")

# --- 🧴 3. MISSING VALUE HYGIENE CORRECTION ---
print("\n🩹 Auditing missing values...")
null_counts = df_cleaned.isnull().sum()
columns_with_nulls = null_counts[null_counts > 0]

if not columns_with_nulls.empty:
    for col, count in columns_with_nulls.items():
        print(f"    - Found {count} null values in '{col}'. Applying median imputation")
        # Imputing with median to safeguard against extreme statistical outliers
        df_cleaned[col] = df_cleaned[col].fillna(df_cleaned[col].median())
else:
```

```

    print("    - No missing values detected in the feature set.")

# --- 🧠 4. COMPOSITE FEATURE ENGINEERING ---
print("\n🔗 Engineering domain-specific composite metrics...")

# Feature 1: Points Per Minute (PPM) - normalizes scoring frequency across variables
# Adding a small epsilon denominator protection value to avoid division-by-zero errors
df_cleaned['pts_per_min'] = df_cleaned['pts'] / (df_cleaned['min'] + 1e-5)

# Feature 2: Player Box Efficiency Rating (PBER) - combines positive and negative contributions
df_cleaned['player_efficiency'] = (
    (df_cleaned['pts'] + df_cleaned['reb'] + df_cleaned['ast'] + df_cleaned['stl'] + df_cleaned['blk']) /
    (df_cleaned['fga'] - df_cleaned['fgm'] + df_cleaned['fta'] - df_cleaned['ftm']) + 1
)

print(f"    - Completed engineering for 'pts_per_min' and 'player_efficiency'."

# --- 📊 5. MULTICOLLINEARITY CORRELATION AUDIT ---
print("\n🔍 Analyzing internal feature correlation structures...")
# Isolate feature matrix excluding target for pure col-to-col assessment
feature_matrix = df_cleaned.drop(columns=[target_col])
corr_matrix = feature_matrix.corr().abs()

# Generate visual heatmap for GitHub/Notebook documentation
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=False, cmap="Blues", cbar=True)
plt.title("NBA Feature Space Absolute Correlation Heatmap", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()

# Find feature pairs with a high correlation coefficient (> 0.850)
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
highly_correlated = [column for column in upper_tri.columns if any(upper_tri[column] > 0.85)]

print(f"⚠️ Highly Multi-Collinear Predictor Pairs Identified (>0.85):")
print(f"    {highly_correlated}")

# --- 🎯 6. VERIFIED FINAL REPO MATRIX STATISTICS ---
print("\n=== 📋 VERIFIED SUMMARY METRICS FOR YOUR SUBMISSION ===")
print(f"Final Data Shape Ready for ML Models: {df_cleaned.shape[0]} rows x {df_cleaned.shape[1]} columns")
print(f"Engineered 'pts_per_min' Median      : {df_cleaned['pts_per_min'].median():.2f}")
print(f"Engineered 'player_efficiency' Mean: {df_cleaned['player_efficiency'].mean():.2f}")
print(f"Class Distribution Baseline (Spans 5 Yrs) : {dict(df_cleaned[target_col].value_counts().sort_index())}")

```


📖 Loading data from 'ReadMe.csv'...

✔ Source Data Loaded: 1340 players, 22 raw attributes.



```
- Removed columns: ['name', 'Unnamed: 0']
```

- Isolate target variable: 'target_5yrs'

- No missing values detected in the feature set.

- Completed engineering for 'pts_per_min' and 'player_efficiency'.

A line graph on a 5x5 grid. The line starts at (0, 5), goes down to (1, 4), up to (2, 4.5), and then down to (3, 3).

